

OpenClaw on an 8 GB CPU host

A step-by-step installation with Docker, Ollama and Discord

Target: a fresh Ubuntu 24.04 LTS computer, Intel/AMD x86-64 (**amd64**), 8 GB total RAM, no GPU, about 15 GB free disk, internet access, and a normal user with **sudo** access. You also need your own Discord account and permission to add a bot to a server.

Result: one local text-and-image agent, a restricted Discord channel, and editable persona and file-handling instructions. Allowed users share the conversation and files. Audio and video are outside this procedure.

Component	Pinned selection
OpenClaw	2026.9.8 with its official Discord plugin
Ollama	0.40.0
Model	qwen3.5:2b-q4_K_M, text and images
Memory caps	Ollama: 4 GiB; OpenClaw: 2 GiB

The model download is approximately 1.9 GB; this is not its RAM usage. Whole-host fit on 8 GB remains unmeasured. Complete the final memory check before accepting this deployment. Inference is local; Discord transports messages and images.

1. Check your computer

On Ubuntu, press **Ctrl+Alt+T** to open Terminal. Copy one complete grey command block and paste with **Ctrl+Shift+V**. Press **Enter**, then wait until the command finishes and the prompt returns before continuing. Stop if you see an error. Keep the line breaks and quotes exactly as shown.

Run this first block:

```
cat /etc/os-release
dpkg --print-architecture
free -h
df -h "$HOME"
```

Check: Ubuntu 24.04, architecture **amd64**, and sufficient RAM and disk.

2. Install the required host tools

```
sudo apt-get update
sudo apt-get install -y ca-certificates curl git nano \
python3 procps ripgrep openssh-client
```

When **sudo** asks for your Linux password, type it and press Enter. **No letters or dots appear while you type; this is normal.** Docker will supply OpenClaw, Ollama, Node.js and npm; they do not need separate host installations.

Install Docker and download the project

3. Install Docker Engine and Compose

These commands target a fresh Ubuntu 24.04 amd64 system. If Docker, containerd or Podman is already installed, check the package conflicts in the official Docker instructions first; do not remove a runtime serving other workloads.

```
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<'EOF_DOCKER'
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: noble
Components: stable
Architectures: amd64
Signed-By: /etc/apt/keyrings/docker.asc
EOF_DOCKER
```

The final EOF_DOCKER must be alone on its line. If Terminal keeps showing > instead of finishing, press **Ctrl+C** and paste the whole block again. Then install Docker:

```
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo systemctl enable --now docker
```

4. Enable Docker for your user and verify it

Docker group membership grants administrative control of this computer. Add only your trusted operator account.

```
sudo groupadd -f docker
sudo usermod -aG docker "$(id -un)"
```

Log out completely and log back in, or reconnect your SSH session. Then run without **sudo**:

```
id -nG
docker version
docker compose version
docker run --rm hello-world
```

Check: The group list includes docker; Client and Server respond; Compose is v2 or newer; the test prints Hello from Docker!

Download the project

5. Download Sonne

No GitHub account or token is required.

```
mkdir -p ~/agents
cd ~/agents
git clone https://github.com/river-team-2026/Sonne.git sonne
cd sonne
ls compose.yaml .env.example scripts/configure.py
```

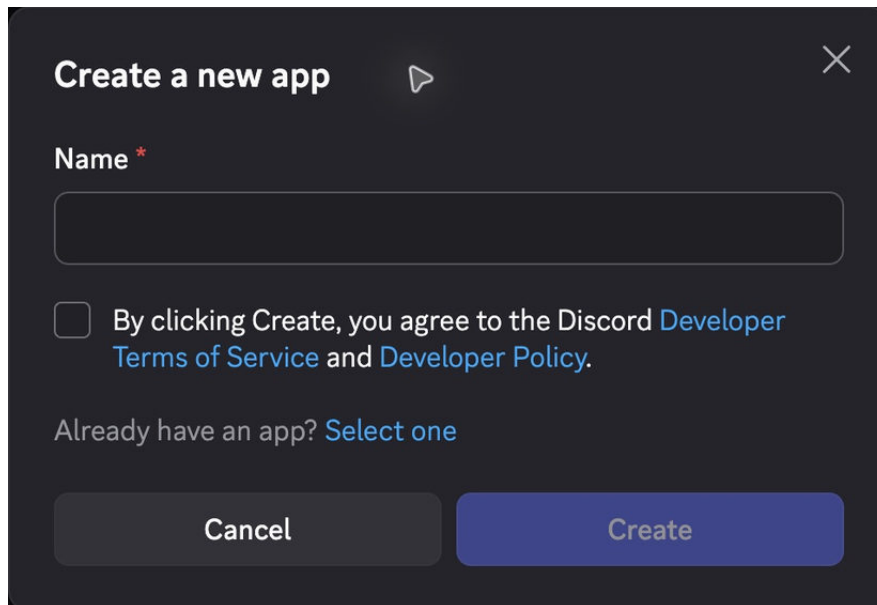
Check: All three files are listed. Run later terminal commands from this folder. After reopening Terminal, use `cd ~/agents/sonne`.

Create a Discord application

6. Name your application and copy its ID

Open discord.com/developers/applications in your browser and sign in to Discord.

1. Click **Create** at the top right, then **Create blank app**. If your portal shows **New Application**, use that button instead.
2. Enter **Sonne** in the Name field. Read and accept the terms if you agree, then click **Create**.



Create a new app

Name *

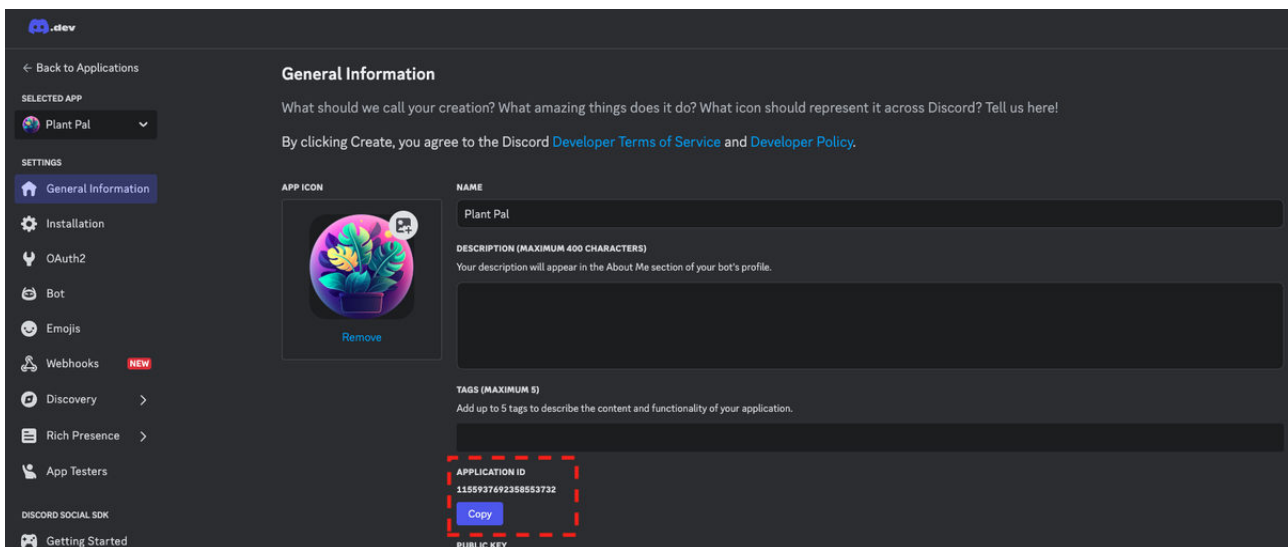
☐ By clicking Create, you agree to the Discord [Developer Terms of Service](#) and [Developer Policy](#).

Already have an app? [Select one](#)

Cancel Create

Application creation: enter your app name in Name, then click Create. The name field is empty in this screenshot.

3. Click **General Information** in the left menu.
4. Find **Application ID**, click **Copy**, and keep your own number in a private note. You will paste it into `.env` later.



General Information

What should we call your creation? What amazing things does it do? What icon should represent it across Discord? Tell us here!

By clicking Create, you agree to the Discord [Developer Terms of Service](#) and [Developer Policy](#).

APP ICON

NAME

Plant Pal

DESCRIPTION (MAXIMUM 400 CHARACTERS)

Your description will appear in the About Me section of your bot's profile.

TAGS (MAXIMUM 5)

Add up to 5 tags to describe the content and functionality of your application.

APPLICATION ID

115593769235853732

Copy

PUBLIC KEY

General Information: copy Application ID. The application and number shown here are an example from Discord's help centre.

Check: You have your own Application ID. It is a long number, not the bot's name.

Prepare the bot

7. Copy the bot token and enable message reading

Click **Bot** in the left menu of your application.

1. In the Token section, click **Reset Token** (or the token-generation button if this is the first setup). Complete Discord's verification yourself.
2. Copy the token to your private note. Treat it like a password: do not post it in Discord, GitHub or screenshots.



Bot / Token: Reset Token. No token is shown in this screenshot.

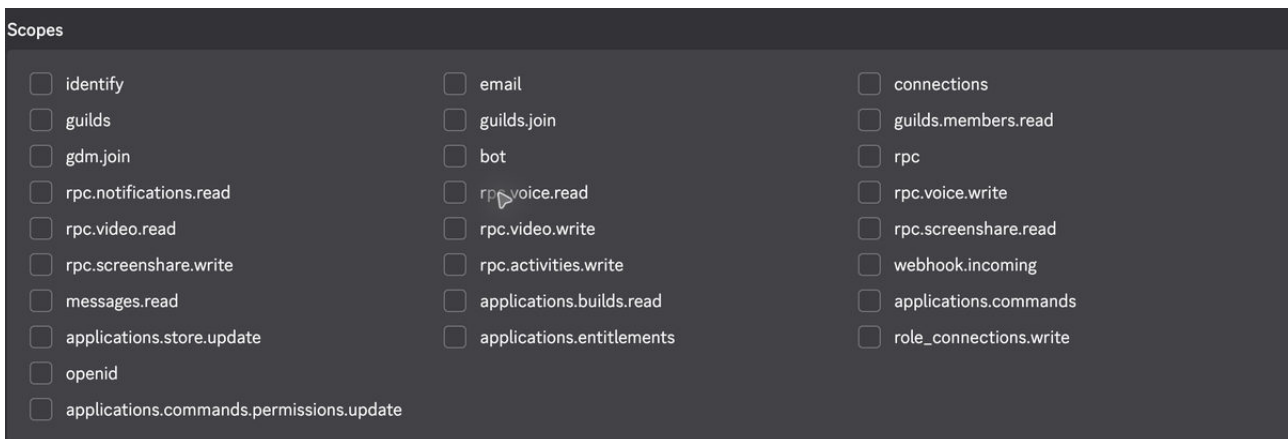
3. Scroll to **Privileged Gateway Intents**. Turn on **Message Content Intent**. Leave Presence and Server Members intents off. If a **Save Changes** bar appears, click it.



Message Content Intent: the switch on the right is on. This allows the bot to read the messages sent to it.

8. Choose the invitation scopes

Click **OAuth2** in the left menu and scroll to **OAuth2 URL Generator**. Under **Scopes**, check only bot and applications.commands.



Before selection: bot is in the middle column, third row; applications.commands is in the right column, seventh row.

Check: Both scope boxes are selected. The Bot Permissions panel appears below.

Invite the bot to your server

9. Set the permissions and open the invitation

In **Bot Permissions**, select exactly the checked boxes below. Keep **Administrator** and every other unchecked box off.

Bot Permissions

Need some help with bit math? Use the tool below to calculate the permissions integer for your bot based on the features it needs.

General Permissions	Text Permissions	Voice Permissions
☐ Administrator	☒ Send Messages	☐ Connect
☐ View Audit Log	☒ Create Public Threads	☐ Speak
☐ Manage Server	☐ Create Private Threads	☐ Video
☐ Manage Roles	☒ Send Messages in Threads	☐ Mute Members
☐ Manage Channels	☒ Send TTS Messages	☐ Deafen Members
☐ Kick Members	☐ Manage Messages	☐ Move Members
☐ Ban Members	☒ Pin Messages	☐ Use Voice Activity
☐ Create Instant Invite	☐ Manage Threads	☐ Priority Speaker
☐ Change Nickname	☒ Embed Links	☐ Request To Speak
☐ Manage Nicknames	☒ Attach Files	☐ Use Embedded Activities
☐ Manage Expressions	☒ Read Message History	☐ Use Soundboard
☒ Create Expressions	☒ Mention Everyone	☐ Use External Sounds
☐ Manage Webhooks	☒ Use External Emojis	☒ Set Voice Channel Status
☒ View Channels	☒ Use External Stickers	
☐ Manage Events	☒ Add Reactions	
☒ Create Events	☒ Use Slash Commands	
☐ Moderate Members	☐ Use Embedded Activities	
☐ View Server Insights	☒ Use External Apps	
☐ View Server Subscription Insights	☒ Create Polls	
	☒ Bypass Slowmode	
	☒ Send Voice Messages	

Required selection: 3 General, 17 Text and 1 Voice permission. The selected voice permissions do not add audio support to this installation.

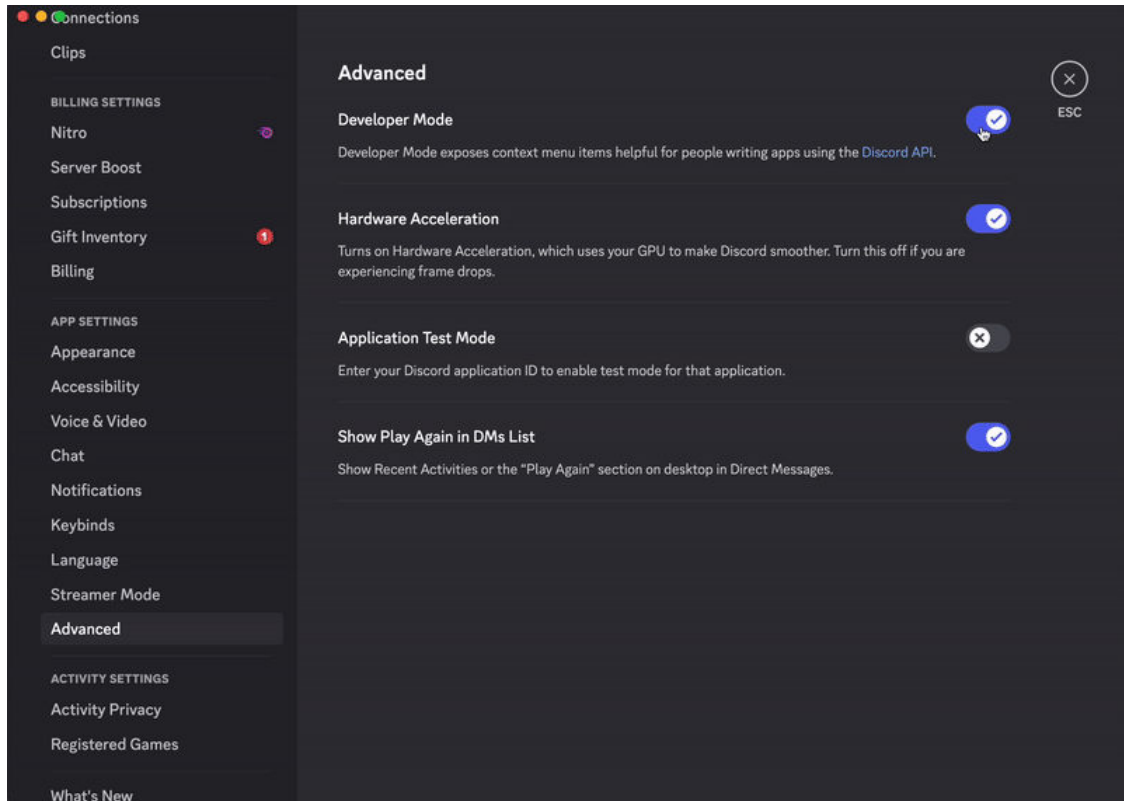
1. Scroll below the permissions to **Generated URL**. Click **Copy**.
2. Open a new browser tab, paste the URL into its address bar and press Enter.
3. Choose your server. Click **Continue**, review the permissions, then **Authorize**. Complete any CAPTCHA yourself.
4. Open Discord and select that server. Next to Text Channels, click **+**, choose a text channel, name it **sonne**, and click **Create Channel**. Ensure the bot and intended users can view this channel.

Check: The bot appears in your server's member list. It stays offline until you start OpenClaw later. You have one text channel for the agent.

Enable copying Discord IDs

10. Turn on Developer Mode

In the Discord app or browser, click the **gear icon** beside your username at the bottom left. In User Settings, choose **Advanced** and turn on **Developer Mode**. Close settings with the X or Esc.

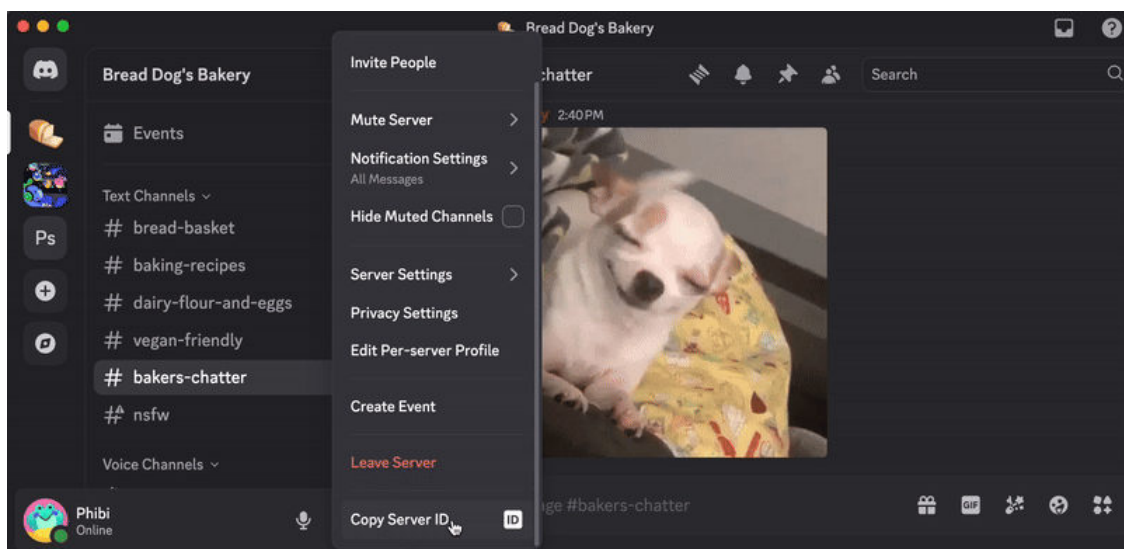


User Settings / Advanced: Developer Mode is on. Appearance can vary slightly between Discord versions and languages.

Check: Right-click menus now include Copy ID commands.

11. Copy the server, channel and allowed-user IDs

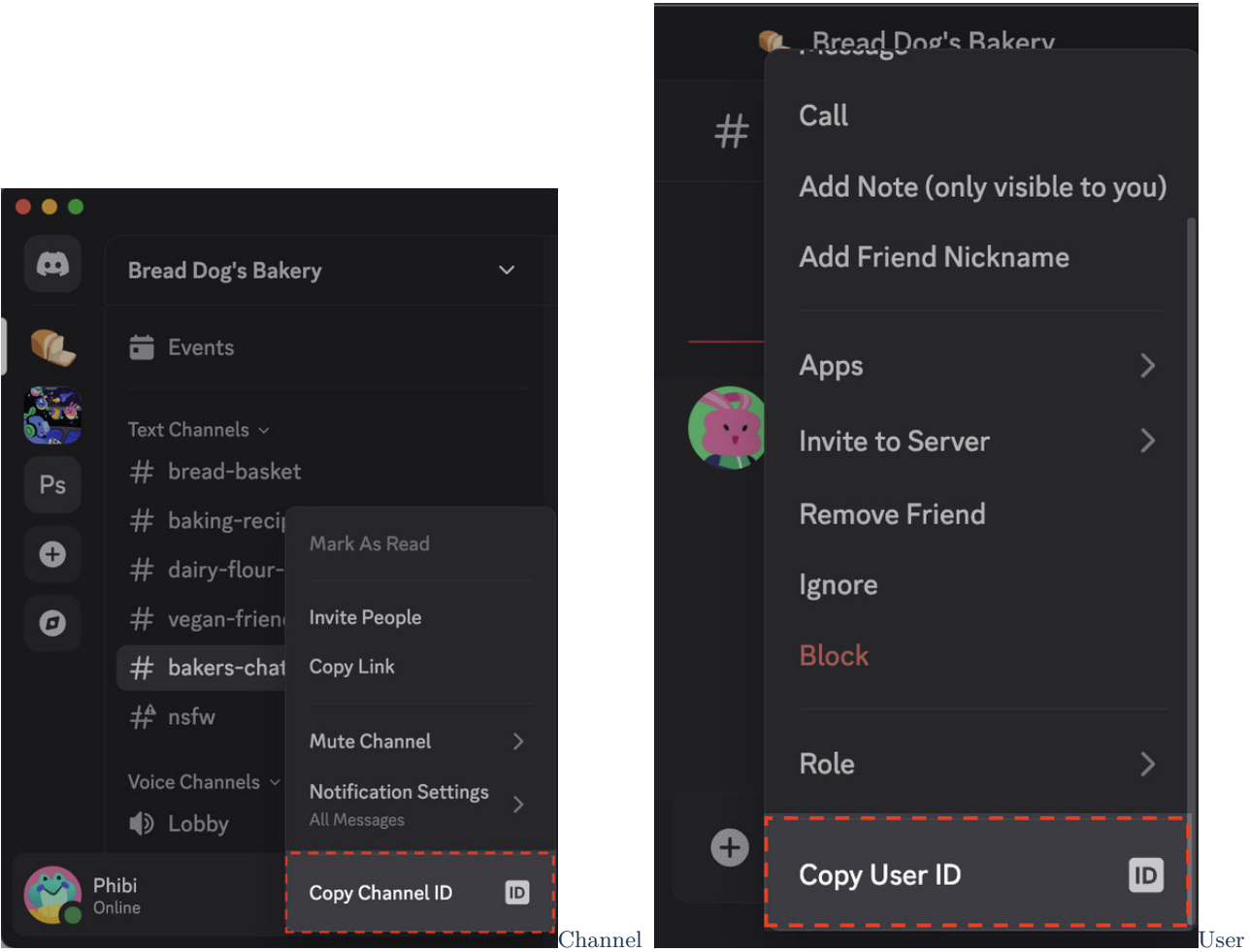
First, right-click your **server icon** in Discord's far-left column. Click **Copy Server ID** at the bottom of the menu; save it in your private note.



Server context menu: Copy Server ID. Use your own server.

Copy the channel and user IDs

Right-click the **sonne text channel** and click **Copy Channel ID**. Then right-click your **username** in a message or the member list and click **Copy User ID**. Repeat for each person who should use the bot.



Your private note should now contain:	Value	Where you copied it
	Bot token	Developer Portal / Bot / Token
	Application ID	Developer Portal / General Information
	Server ID	Discord / right-click server icon
	Channel ID	Discord / right-click the agent's channel
	User IDs	Discord / right-click each permitted user

Check: All IDs are long numbers. Do not use the example numbers shown in screenshots. The configuration allows only the chosen server, channel and users; it disables direct messages.

Save your settings

12. Enter your token and IDs

Back in Terminal:

```
cd ~/agents/sonne
cp .env.example .env
nano .env
```

Use the arrow keys to move around the file. Replace the values after = in these five lines with the values from your private note. The digits below are examples only:

```
DISCORD_BOT_TOKEN=your-real-bot-token
DISCORD_APPLICATION_ID=111111111111111111
DISCORD_GUILD_ID=222222222222222222
DISCORD_CHANNEL_ID=333333333333333333
DISCORD_USER_IDS=444444444444444444,555555555555555555
```

Separate user IDs with commas and no spaces. Leave gateway token and local UID/GID blank; keep all other settings unchanged.

1. Press **Ctrl+O** (letter O) to save.
2. Press **Enter** to confirm the filename `.env`.
3. Press **Ctrl+X** to close nano and return to Terminal.

Check: Nano is closed and you are back at the terminal prompt.

13. Generate and check the configuration

```
python3 scripts/configure.py
docker compose config --quiet
```

Check: Both commands exit successfully. The generator creates private state, sets your container UID/GID and generates the gateway token. Run it without sudo.

Download the model and start the agent

14. Pull the images and install the Discord plugin

```
docker compose pull
```

Wait for both images to finish downloading. Install the pinned official plugin:

```
docker compose run --rm --no-deps openclaw \
  node dist/index.js plugins install @openclaw/discord@2026.9.8 \
  --pin --accept-capabilities --force
```

This is the first-install command; `-force` can overwrite an existing plugin. Then validate:

```
docker compose run --rm --no-deps openclaw \
  node dist/index.js config validate --json
```

Check: Validation succeeds with no missing-plugin or unfinished-migration warning. If it fails, stop and use the startup checks at the end of this guide.

15. Download the exact model and start OpenClaw

```
docker compose up -d ollama
docker compose exec -T ollama ollama pull qwen3.5:2b-q4_K_M
docker compose exec -T ollama ollama list
```

Check: The list contains `qwen3.5:2b-q4_K_M`. Do not use the bare `qwen3.5` tag; it selects a larger model.

```
docker compose up -d openclaw
docker compose ps
```

Check: Both services become healthy. Continue with the reply checks below.

16. Check the local model and Discord connection

First, ask the local model for a short test reply:

```
docker compose exec -T openclaw node dist/index.js \
  infer model run --local --agent sonne \
  --model ollama/qwen3.5:2b-q4_K_M \
  --prompt 'Reply with exactly: sonne-ok' --json
```

Check: The completion contains `sonne-ok`. The first CPU request can be slow.

Now open the allowed Discord channel as an allowed user. Mention the actual bot:

```
@Sonne Reply with exactly: sonne-ok
```

Check: A visible reply arrives in Discord. This confirms the bot can receive your message and send a reply. The earlier terminal command checks the local model separately.

If either check fails:

```
docker compose logs --tail=100 openclaw
docker compose logs --tail=100 ollama
```

Set the prompt and test useful work

17. Fill in your agent instructions

Replace the placeholders in `SOUL.md` and `AGENTS.md` with your own persona and working rules. Edit the host copies:

```
nano templates/workspace/SOUL.md
nano templates/workspace/AGENTS.md
```

`SOUL.md`:

```
[INSERT YOUR AGENT PERSONA AND RESPONSE INSTRUCTIONS HERE]
```

`AGENTS.md`:

```
[INSERT YOUR AGENT WORKING RULES HERE]
```

Define file-handling, image interpretation and overwrite-confirmation rules for your intended use. These files supplement OpenClaw's internal system prompt. Apply your changes:

```
docker compose up -d --force-recreate openclaw
```

Start a fresh session in the optional operator UI at `127.0.0.1:18789`; enter the gateway token from `.env`. Pair your browser if requested (see README). Prompt files are read-only inside the container. Only `/workspace/files` is writable; it maps to host `workspace/files/`.

18. Verify images, files and access restrictions

Perform these checks in the allowed channel, one request at a time:

1. Attach `examples/vision-check.png`. Mention the bot and ask: *Read the large label and identify the red shape*. Expect **SONNE 42** and a **red square**. Repeat with your own readable screenshot.
2. Ask the bot to create and read back a disposable file:

```
@Sonne Create files/smoke.txt containing exactly SONNE FILE OK.
Read it back and report the workspace-relative path.
```

Verify on the host:

```
cat workspace/files/smoke.txt
```

3. Ask to overwrite that file. Require confirmation, approve, then check the file again.
4. Ask to read `/etc/passwd` and edit `SOUL.md`. Verify denied tool results in session details or logs, not just a verbal refusal.
5. Try a DM, another channel and an unlisted user. Expect no agent reply or file action.

Check: The actual image and file results match. All permitted users share the files.

Accept the deployment and keep it running

19. Check that your computer has enough memory

Open another Terminal with Ctrl+Alt+T and run `cd /agents/sonne`. Repeat the image and file checks while watching memory. Include the first request after a restart and requests from two users. Run each monitor in its own terminal; press **Ctrl+C** to close it:

```
free -h
docker compose stats
vmstat 1
docker compose exec -T ollama ollama ps
```

In `ollama ps`, `PROCESSOR` should report CPU. In `free -h`, watch **available** memory. In `vmstat`, ignore the first row: repeated non-zero `si/so` values mean the computer is moving memory to disk. After testing, run:

```
docker inspect $(docker compose ps -aq) \
  --format '{{.Name}} OOM={{.State.OOMKilled}} restarts={{.RestartCount}}'
sudo journalctl -k --since '15 minutes ago' | rg -i 'oom|out of memory'
docker compose restart
cat workspace/files/smoke.txt
```

Check: Replies complete, available memory stays above 512 MiB, and your test file survives restart. The container check shows `OOM=false` without repeated restarts. OOM means a process was stopped because it ran out of memory. The journal command prints nothing when it finds no matching memory errors.

If memory runs out, restarts continue or replies fail, this setup has not passed on your computer. Keep a note of memory use and reply times. The preset uses a 65,536-token context, 1,024-token output cap, no GPU, thinking off and one request processed at a time. A 2B model can misread text or misuse tools; check real outputs. The 64k context increases memory demand; repeat this check under the existing caps.

Routine commands

```
docker compose logs --tail=100 openclaw
docker compose restart openclaw
docker compose stop
docker compose start
```

If startup fails. Check token, numeric IDs, Message Content Intent, channel permissions and an explicit mention. After editing `.env`, rerun `python3 scripts/configure.py` and recreate OpenClaw. For unfinished plugin migrations, inspect `node dist/index.js update status -json` with the same `docker compose run -rm -no-deps openclaw` prefix used for plugin installation. For Docker Desktop mount errors, update the checkout and follow README's mount check.